

Sistema Censal INEGI - Prueba Técnica (Docker + Django + PostgreSQL/PostGIS)

Consola web ejecutiva y plataforma de análisis de datos demográficos sobre el esquema IDTEST de PostgreSQL. El proyecto integra un pipeline ETL en Python, scripts de procedimientos almacenados (DQL), entornos contenedorizados con Docker Compose y un dashboard web responsivo desarrollado en Django.

Arquitectura y Tecnologías

- Backend Framework:** Django 6.x (Python 3.12+)
 - Base de Datos:** PostgreSQL 16 con extensión PostGIS (postgis/postgis:16-3.4)
 - Pipeline ETL:** Python (Pandas, Psycopg2)
 - Infraestructura de servidores:** Docker & Docker Compose
 - Frontend:** Bootstrap 5, Bootstrap Icons, Chart.js
-

Estructura del Proyecto

```
.
├── census/          # Aplicación principal Django (Lógica de negocio e interfaz)
│   ├── templates/census/ # Plantillas HTML (Home e Incisos A, B y C)
│   ├── models.py       # Modelos de datos para el esquema idtest
│   ├── services.py     # Capa de servicio para invocación de Stored Procedures
│   ├── views.py        # Controladores y lógica de vistas
│   └── urls.py         # Enrutamiento interno de la aplicación census
├── core/            # Configuración del proyecto Django (settings, urls, wsgi)
├── docs/            # Documentación oficial generada y entregables en PDF/ODT
│   ├── PRUEBA_TECNICA_BD.pdf# Respuestas y notas a la prueba técnica
│   └── README.pdf      # Guía e instrucciones de ejecución en PDF
├── dql/             # Scripts SQL de Stored Procedures (sp_a, sp_b, sp_c)
├── etl/             # Pipeline ETL para procesamiento e ingesta de datos
│   ├── DATA/         # Archivos CSV transformados y Dumps de la BD (.sql y .dump)
│   ├── etl_down_up.py  # Script principal de Extracción, Transformación y Carga masiva (COPY)
│   └── scan_csv.py     # Script de inspección y análisis del dataset original
```

```
|   |─ aux.py          # Mapeo de columnas y configuraciones auxiliares del ETL
|   └─ BD_PRUEBA_.csv  # Fuente de datos original (dataset base)
|─ DDL_IDTEST.sql      # Definición DDL del esquema idtest, tablas e índices
|─ docker-compose.yml  # Orquestación de contenedores (App Django + PostgreSQL)
|─ Dockerfile          # Configuración del contenedor para la aplicación web
|─ manage.py           # Gestor de comandos de Django
|─ README.md           # Documentación principal en Markdown
└─ requirements.txt     # Dependencias del proyecto en Python
```

Requisitos Previos

Asegúrate de contar con las siguientes herramientas instaladas en tu sistema:

- [Git](#)
 - [Docker](#) y [Docker Compose](#)
-

Pasos para Clonar y Ejecutar el Proyecto

1. Clonar el Repositorio

```
git clone https://github.com/brandhi/idtest.git
cd idtest
```

Es importante agregar un archivo `.env` en la raíz del proyecto con los siguientes datos de prueba

```
# Django Settings
DEBUG=True
SECRET_KEY="35*$9_@o@w+dy!5i5sp9%*oi)9cn80&w(4xriw#d@oa^4&dy*$"

# PostgreSQL Credentials
DB_NAME=idtest_db
DB_USER=postgres
DB_PASSWORD=postgres_password
DB_HOST=127.0.0.1
DB_PORT=5432
```

Brandon Jimenez Rodriguez

2. Levantar el Entorno con Docker Compose

Este comando compilará la aplicación web Django e inicializará el contenedor de PostgreSQL/PostGIS montando automáticamente la estructura DDL y los Stored Procedures (/docker-entrypoint-initdb.d/).

```
docker compose up -d --build
```

Verifica que ambos contenedores estén corriendo correctamente:

```
docker compose ps
```

Carga de Datos y Restauración de la Base de Datos

Tienes **dos opciones** para poblar la base de datos con la información censal:

Opción A: Ejecutar el Pipeline ETL (Recomendado)

El script `etl_down_up.py` procesa el CSV original (`BD_PRUEBA_.csv`), genera los archivos limpios en `./etl/DATA/`, realiza la carga masiva mediante `COPY` en PostgreSQL y exporta los dumps actualizados.

Para ejecutarlo dentro del entorno virtual o desde el host (asegurándote de que la BD está activa en el puerto 5432):

Opción dentro del contenedor de Docker (sin instalar Python en la máquina host):

```
docker compose exec web python etl/etl_down_up.py
```

O si se prefiere ejecutar desde el host (requiere dependencias locales):

```
pip install -r requirements.txt
```

```
python etl/etl_down_up.py
```

Opción B: Restaurar desde el Data Dump (.sql o .dump)

Si prefieres restaurar la base de datos directamente usando el dump ubicado en `./etl/DATA/`:

Restaurar usando el archivo .sql (Texto Plano):

```
docker exec -i idtest_db psql -U postgres -d idtest_db < etl/DATA/idtest_db_dump.sql
```

Restaurar usando el archivo .dump (Custom Format):

```
docker exec -i idtest_db pg_restore -U postgres -d idtest_db -n idtest --clean --if-exists < etl/DATA/idtest_db_dump.dump
```

Acceso a la Aplicación Web

Una vez que los contenedores estén arriba y la base de datos poblada, abre tu navegador e ingresa a:

<http://localhost:8000/census/>

Nota: Esta es una URL local, solo funcionara en la maquina host en la que se ejecute el proyecto.

Módulos Disponibles:

- **Dashboard Principal (/):** Vista general con accesos directos, esquema de la base de datos y enlace de descarga para los 5 archivos ETL/Dump.
 - **Inciso A (/inciso-a/):** Consulta de la Población Femenina de 15 a 49 años (SP_GET_FEMALE_POPULATION).
 - **Inciso B (/inciso-b/):** Análisis de Métricas de Discapacidad por Estado (SP_GET_DISABILITY_BY_STATE).
 - **Inciso C (/inciso-c/):** Comparativo de Datos Demográficos Absolutos (SP_GET_DEMOGRAPHICS).
-

Detener o Reiniciar el Entorno

Para detener los servicios:

```
docker compose down
```

Para reiniciar borrando el volumen de la base de datos (recreación desde cero):

```
docker compose down -v
```

```
docker compose up -d
```